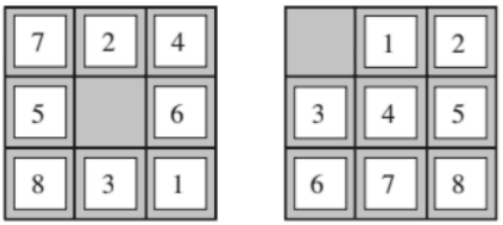
U24AI006

ASSIGNMENT -02

AI202 Artificial Intelligence
Lab Assignment 2

Q1 The following figure shows the start state and goal state for the 8-puzzle problem. Find the number of states explored before reaching the goal state using the BFS algorithm.



Start State Goal State

Q2 Find the number of states explored for the DFS algorithm. Give comparison of BFS and DFS in terms of cost if the path cost for the state is given as depth of the search tree at which the state is occurring.

CODE:

```
from collections import deque
import time

# --- Helper Functions ---

def print_board(board):
    """Prints the 3x3 board in a readable format."""
    for row in board:
        print(" | ".join(str(x) if x != 0 else "_" for x in row))
    print("-" * 9)

def find_blank(board):
    """Finds the position (row, col) of the blank space (represented by 0)."""
    for r in range(3):
        for c in range(3):
            if board[r][c] == 0:
                return r, c
    return None

def get_neighbors(state):
    """
    Generates all possible next states from the current state.
    Returns a list of valid board configurations.
    """
```

```
    neighbors = []
    blank_r, blank_c = find_blank(state)

    # Possible moves: Up, Down, Left, Right
    # (row_change, col_change)
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        new_r, new_c = blank_r + dr, blank_c + dc

        # Check if the new position is inside the 3x3 grid
        if 0 <= new_r < 3 and 0 <= new_c < 3:
            # Create a copy of the board to modify
            # (We convert to list of lists to be mutable)
            new_board = [list(row) for row in state]

            # Swap the blank space with the target tile
            new_board[blank_r][blank_c], new_board[new_r][new_c] = \
            new_board[new_r][new_c], new_board[blank_r][blank_c]

            # Convert back to tuple of tuples so it can be stored in a set
            neighbors.append(tuple(tuple(row) for row in new_board))

    return neighbors

def bfs_solver(start_state, goal_state):
    """
    Solves the 8-puzzle using Breadth-First Search (BFS).
    Returns the number of states explored and the depth (moves) of the solution.
    """
    # Queue stores: (current_board_state, depth_level)
    queue = deque([(start_state, 0)])

    # Visited set stores states we have already seen to avoid loops
    # Using a set is crucial for performance!
    visited = set()
    visited.add(start_state)

    states_explored_count = 0

    print("BFS Algorithm Started...")
    print("Searching for the goal (this might take a few seconds)...")

    start_time = time.time()
```

```
    while queue:
        current_state, depth = queue.popleft()
        states_explored_count += 1

        # Check if we have reached the goal
        if current_state == goal_state:
            end_time = time.time()
            print(f"\nSUCCESS! Goal found.")
            print(f"Time taken: {end_time - start_time:.4f} seconds")
            return states_explored_count, depth

        # Get all valid next moves
        next_states = get_neighbors(current_state)

        for state in next_states:
            if state not in visited:
                visited.add(state)
                queue.append((state, depth + 1))

    return -1, -1 # Should not happen if a solution exists

# --- Main Execution Block ---

if __name__ == "__main__":
    # 0 represents the empty/blank space

    # Start State from your image
    # 7 2 4
    # 5 0 6
    # 8 3 1
    start_matrix = (
        (7, 2, 4),
        (5, 0, 6),
        (8, 3, 1)
    )

    # Goal State from your image
    # 0 1 2
    # 3 4 5
    # 6 7 8
    goal_matrix = (
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8)
    )
```

```
print("--- Start State ---")
print_board(start_matrix)
print("\n--- Goal State ---")
print_board(goal_matrix)
print("\n-------------------")

explored, moves = bfs_solver(start_matrix, goal_matrix)

print("\n--- Final Results ---")
print(f"Total States Explored (Nodes visited): {explored}")
print(f"Minimum Moves Required (Depth): {moves}")
```

OUTPUT:

```
 2 QUESTION 1.py"
--- Start State ---
7 | 2 | 4
5 | _ | 6
8 | 3 | 1
---------

--- Goal State ---
_ | 1 | 2
3 | 4 | 5
6 | 7 | 8
---------

-------------------
BFS Algorithm Started...
Searching for the goal (this might take a few seconds)...

SUCCESS! Goal found.
Time taken: 1.1208 seconds

--- Final Results ---
Total States Explored (Nodes visited): 171712
Minimum Moves Required (Depth): 26
```

CODE (2):

```
import collections

class SearchAlgorithmAnalyzer:
    def __init__(self):
        self.graph = {
            'A': ['B', 'C'],
            'B': ['D', 'E'],
            'C': ['F'],
            'D': [],
            'E': ['G'],
            'F': [],
            'G': []
        }

    def run_dfs(self, start_node, goal_node):
        print(f"\n--- Running DFS from '{start_node}' to find '{goal_node}' ---")

        stack = [(start_node, 0)]
        visited = set()
        states_explored = 0

        while stack:
            current_node, depth = stack.pop()

            if current_node in visited:
                continue

            states_explored += 1
            visited.add(current_node)
            print(f"  Exploring: {current_node} (Depth: {depth})")

            if current_node == goal_node:
                print(f"  >> Goal '{goal_node}' found!")
                return {
                    "algorithm": "DFS",
                    "states_explored": states_explored,
                    "cost": depth,
                    "status": "Found"
                }

            neighbors = self.graph.get(current_node, [])
            for neighbor in reversed(neighbors):
                if neighbor not in visited:
                    stack.append((neighbor, depth + 1))
```

```
        return {"algorithm": "DFS", "status": "Not Found"}

    def run_bfs(self, start_node, goal_node):
        print(f"\n--- Running BFS from '{start_node}' to find '{goal_node}' ---")

        queue = collections.deque([(start_node, 0)])
        visited = set()
        visited.add(start_node)
        states_explored = 0

        while queue:
            current_node, depth = queue.popleft()
            states_explored += 1

            print(f"  Exploring: {current_node} (Depth: {depth})")

            if current_node == goal_node:
                print(f"  >> Goal '{goal_node}' found!")
                return {
                    "algorithm": "BFS",
                    "states_explored": states_explored,
                    "cost": depth,
                    "status": "Found"
                }

            neighbors = self.graph.get(current_node, [])
            for neighbor in neighbors:
                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append((neighbor, depth + 1))

        return {"algorithm": "BFS", "status": "Not Found"}

analyzer = SearchAlgorithmAnalyzer()

start = 'A'
goal = 'G'

dfs_result = analyzer.run_dfs(start, goal)
bfs_result = analyzer.run_bfs(start, goal)

print("\n" + "="*40)
print("       FINAL COMPARISON RESULTS       ")
print("="*40)
```

U24AI006

```
print(f"{'Metric':<20} | {'DFS':<10} | {'BFS':<10}")
print("-" * 46)
print(f"{'States Explored':<20} | {dfs_result['states_explored']:<10} |
{bfs_result['states_explored']:<10}")
print(f"{'Cost (Depth)':<20} | {dfs_result['cost']:<10} |
{bfs_result['cost']:<10}")
print("="*40)

print("\n[Comparison Explanation]")
if bfs_result['cost'] < dfs_result['cost']:
   print(f"BFS found a path with LOWER cost ({bfs_result['cost']}) than DFS
({dfs_result['cost']}).")
   print("This demonstrates that BFS is 'Optimal' for unweighted graphs—it always
finds the shallowest goal.")
elif bfs_result['cost'] == dfs_result['cost']:
   print("Both algorithms found the goal at the same depth.")
else:
   print("DFS happened to find a path, but BFS guarantees the shortest path in
terms of depth.")

print(f"BFS explored {bfs_result['states_explored']} states vs DFS's
{dfs_result['states_explored']} states.")
```

OUTPUT:

```
 2 QUESTION 2.py"

--- Running DFS from 'A' to find 'G' ---
  Exploring: A (Depth: 0)
  Exploring: B (Depth: 1)
  Exploring: D (Depth: 2)
  Exploring: E (Depth: 2)
  Exploring: G (Depth: 3)
  >> Goal 'G' found!

--- Running BFS from 'A' to find 'G' ---
  Exploring: A (Depth: 0)
  Exploring: B (Depth: 1)
  Exploring: C (Depth: 1)
  Exploring: D (Depth: 2)
  Exploring: E (Depth: 2)
  Exploring: F (Depth: 2)
  Exploring: G (Depth: 3)
  >> Goal 'G' found!

========================================
       FINAL COMPARISON RESULTS
========================================
Metric               | DFS        | BFS
----------------------------------------------
States Explored      | 5          | 7
Cost (Depth)         | 3          | 3
========================================

[Comparison Explanation]
Both algorithms found the goal at the same depth.
```